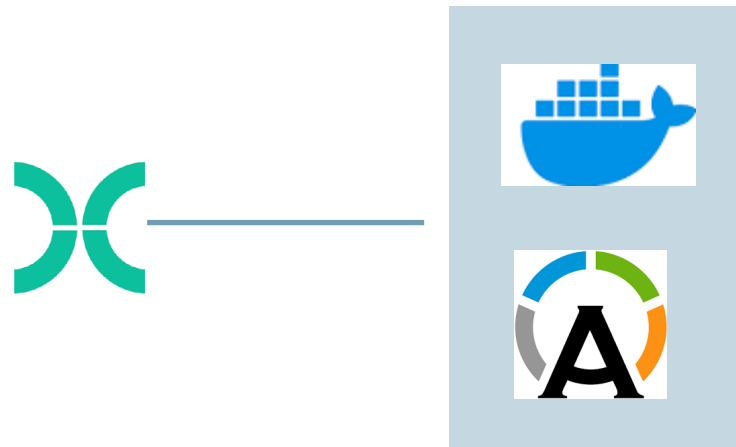
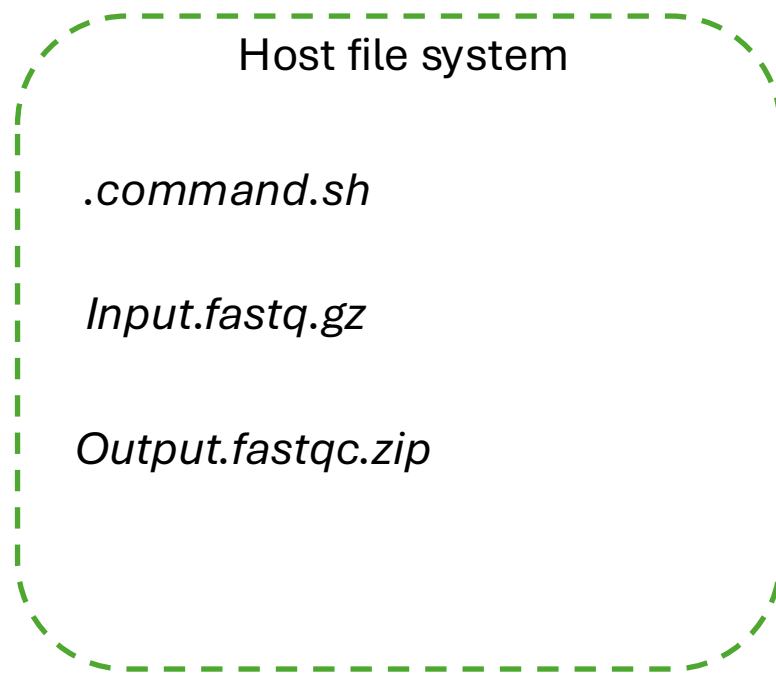
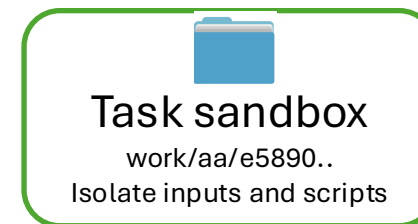
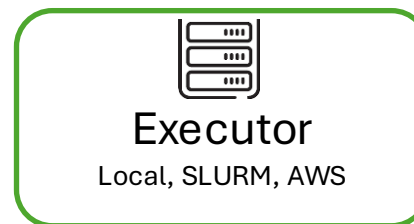
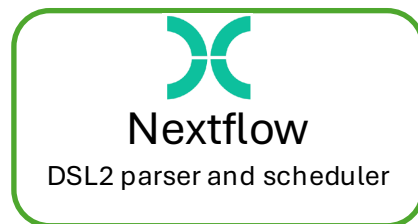


Nextflow Containers Integration for NGS

Orchestrating reproducible workflows with Docker and Apptainer



Execution architecture

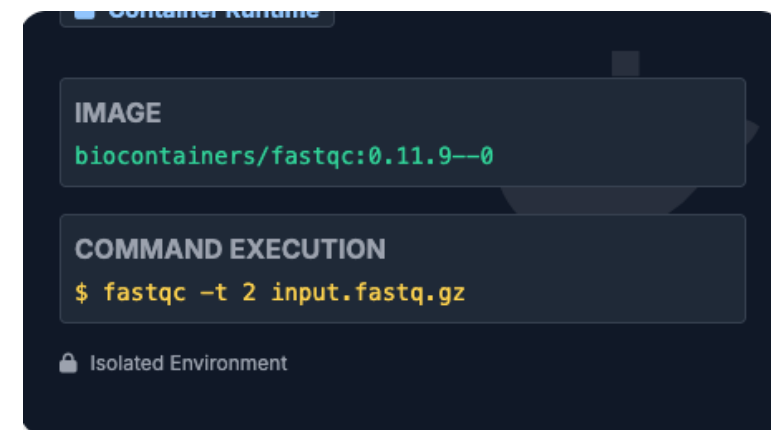


bind mount



Automatically mounts:

- **workDir** (e.g. work/)
- **The project directory** (where main.nf lives)
- **The execution directory** (\$PWD when you launched Nextflow)
- **Any input files** referenced by processes (if outside the above)



Global Configuration



Docker Compatibility

Use **docker://** URIs directly. Nextflow automatically pulls Docker layers and builds a Singularity Image Format (SIF) file on the fly.



Centralized Caching

Define **cacheDir** to store SIF files. This prevents redundant downloads across pipeline runs and different users. If cacheDir is not set Nextflow will automatically save the image in a default folder. For example, for singularity the images are usually in `~/.singularity/cache`



Process Isolation

The **--cleanenv** flag is crucial on HPCs. It prevents host environment modules (like module load ..., PATH, LD_LIBRARY_PATH) from leaking into the container.

```
$ nextflow run main.nf -with-singularity
```

```
$ nextflow run main.nf -with-docker
```

File: **nextflow.config**

```
singularity {
    // Enable Singularity support
    enabled = true
    // mount user's home directory into the container
    autoMounts = true
    // Specify a custom cache directory. Essential for shared environments.
    cacheDir = '/path/to/singularity/cache'
    // Ensure a clean environment for each run
    runOptions = '--cleanenv'
}

process {
    container = 'docker://alpine:3.19'
}
```

File: **sayHello.nf**

```
nextflow.enable.dsl=2

process say_hello {
    // Container defined directly in the process (not in nextflow.config)
    container 'docker://alpine:3.19'

    input:
    val name

    output:
    stdout

    script:
    """
    echo "Hello ${name} from inside the container"
    cat /etc/os-release | head -n 2
    """
}

workflow {
    Channel.of('Alice','Bob') | say_hello
}
```

Global Configuration



Docker Compatibility

Sets a baseline container for any process that doesn't have a specific assignment. Good for basic shell utilities.



Process Isolation

Target a specific process instance. Useful for tools with unique version requirements or heavy resource needs.



Centralized Caching

Apply containers to groups of processes (e.g., all QC tools). Decouples configuration from process logic.

```
$ nextflow run main.nf -with-singularity
```

```
$ nextflow run main.nf -with-docker
```

File: **nextflow.config**

```
singularity {  
    // Enable Singularity support  
    enabled = true  
    // mount user's home directory into the container  
    autoMounts = true  
    // Specify a custom cache directory. Essential for shared environments.  
    cacheDir = '/path/to/singularity/cache'  
    // Ensure a clean environment for each run  
    runOptions = '--cleanenv'  
}  
  
process {  
    container = 'docker://alpine:3.19'  
}
```

File: **sayHello.nf**

```
nextflow.enable.dsl=2  
  
process say_hello {  
    // Container defined directly in the process (not in nextflow.config)  
    container 'docker://alpine:3.19'  
  
    input:  
    val name  
  
    output:  
    stdout  
  
    script:  
    """  
    echo "Hello ${name} from inside the container"  
    cat /etc/os-release | head -n 2  
    """  
}  
  
workflow {  
    Channel.of('Alice','Bob') | say_hello  
}
```

Global Configuration

What happens if you set `autoMounts = false`

Nextflow becomes much more strict/minimal about binds. You usually still get the essentials to run the task, but **it won't go out of its way** to mount extra host paths.

Typical effects you'll notice:

- Your process may fail if it tries to access files via absolute paths that are **not staged** into the work dir and not otherwise mounted.
- Access to pipeline auxiliary files (e.g., bin/ scripts, reference files stored alongside the project) may require that you:
 - stage them as input: paths
 - explicitly add binds via engine options.

Why this option exists

- **Portability / convenience (`autoMounts=true`)**: good for most users and teaching, fewer “file not found” surprises.
- **Reproducibility / security (`autoMounts=false`)**: reduces accidental dependence on arbitrary host directories and reduces “it works on my machine” effects.

Reproducibility & Version Pinning

Pin Exact Image Tags

Never use latest. Always specify explicit version numbers to ensure the same software version is used every run.

Preferred: digest (sha256:...)

Centralize Versions in Params

Define tool versions in nextflow.config params. This allows easy upgrades across the entire pipeline from a single location.

Record Execution Metadata

Always generate execution reports to document exactly which containers and hash versions were used.

Avoid Vendor Lock-in

Support both docker and singularity profiles. Public registries (Quay.io) work for both engines.



Caching & Consistency

The container image hash is part of the task definition. If you change the image tag or digest, Nextflow automatically invalidates the cache and re-runs the step, ensuring correctness.

Final Takeaway

Don't reinvent the wheel. Reuse nf-core modules and pipelines to inherit robust, secure, and reproducible containerization defaults immediately.